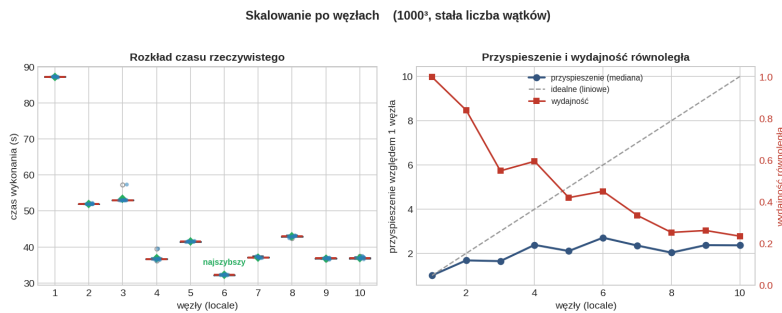


Analiza wydajności heat3d (stencil dyfuzji ciepła 3D, Chapel)

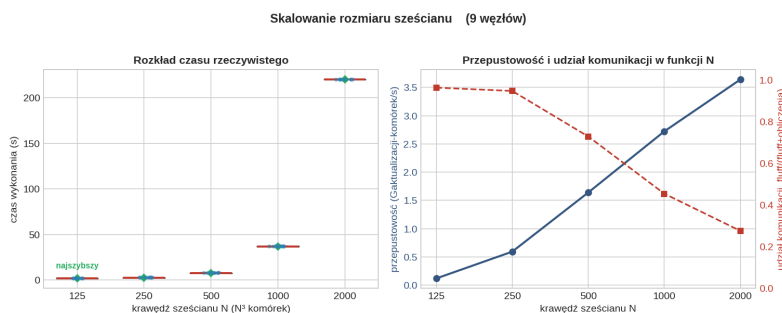
Klaster obliczeniowy, conduit GASNet. **Locale** w Chapelu to **osobny proces** (z własną pamięcią), uruchamiany flagą `-n1 N`; w tych testach jeden locale na węzeł fizyczny. Liczbę *wątków wewnątrz jednego locale* steruje `CHPL_RT_NUM_THREADS_PER_LOCALE` – to dwie niezależne osie zrównoleglenia (locale = między procesami, wątki = w obrębie procesu). Lewy panel każdego rysunku: rozkład czasu wykonania (box-plot, ~ 10 przebiegów/konfigurację; czerwona linia – mediana, zielony diament – średnia, „najszybszy” – najniższa mediana). Prawy panel: metryka pochodna.

1. Skalowanie po węzłach (1000³, stała liczba wątków)



- Czas spada z ~ 87 s (1 węzeł) do minimum ~ 32 s **przy 6 węzłach** i dalej **nie maleje** (7–10 węzłów: ~ 37 s).
- Silny „ząbek piły” (parzysta/nieparzysta liczba węzłów) – podział 1000³ na nierówne płyty daje różne obciążenia i różną liczbę bajtów/krok.
- Przyspieszenie płaszczy się na $\sim 2,4\times$, mocno poniżej idealnej prostej; wydajność równoległa spada do $\sim 0,24$ przy 10 węzłach.
- **Wniosek:** aplikacja jest ograniczona komunikacją (*comm-bound*) – wymiana halo (`updateFluff`) rośnie liniowo z liczbą węzłów, a wolny conduit zjada zysk z rdzeni.

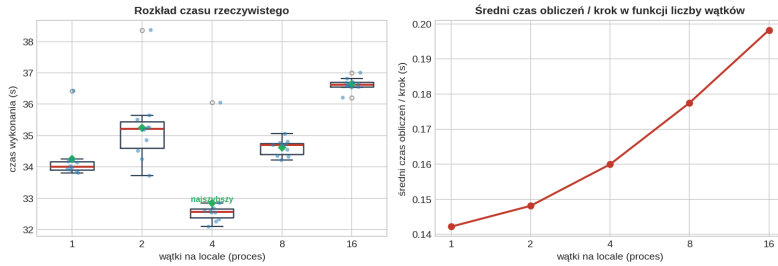
2. Skalowanie rozmiaru sześciangu (9 węzłów)



- Czas rośnie $\sim$ sześciennie z krawędzią N (N^3 komórek): od $\sim 1,7$ s (125³) do ~ 220 s (2000³).
- Przepustowość rośnie z 0,12 do $\sim 3,6$ **Gkom./s** (miliardów aktualizacji komórek na sekundę).
- Udział komunikacji `fluff/(fluff+obliczenia)` spada z 0,96 **do** 0,28.
- **Wniosek:** odwrotność rys. 1 – większy sześciang to korzystniejszy stosunek obliczeń do komunikacji (objętość N^3 rośnie szybciej niż powierzchnia halo N^2). Małe siatki są zdominowane przez sieć i marnują klaster.
- *Uwaga:* przy 2000³ tylko 2 realne przebiegi (reszta dopełniona średnią).

3. Skalowanie wątków przy 9 locale (1000³, 100 kroków)

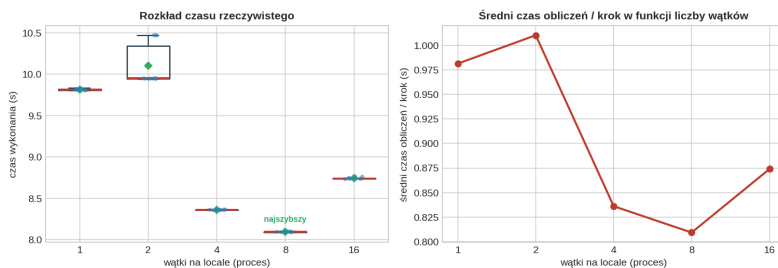
Skalowanie wątków przy 9 locale (1000³, 9 locale, 100 kroków)



- Stałe **9 locale** (9 procesów, po jednym na węzeł); zmienną jest liczba *wątków w obrębie każdego locale* (1 → 16).
- Czas niemal płaski (~33–37 s); minimum przy **4 wątkach/locale** (~32,6 s), potem rośnie.
- Średni czas obliczeń *na krok* rośnie monotonicznie z liczbą wątków (0,14 → 0,20 s).
- **Wniosek:** dokładanie wątków w obrębie procesu nie pomaga – problem jest *comm-bound* (wymiana halo między locale, nie moc obliczeniowa). Wzrost czasu obliczeń/krok to nadsubskrypcja rdzeni: rywalizacja o pasmo pamięci i o wątek pollujący GASNet, który już zajmuje jeden rdzeń w każdym procesie.

4. Skalowanie wątków w obrębie jednego locale (1000³, 10 kroków)

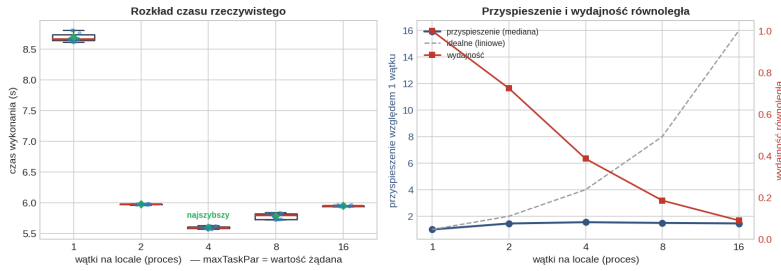
Skalowanie wątków w obrębie jednego locale (1000³, 1 locale, 10 kroków)



- Próba kontrolna: **1 locale** (1 proces), więc **brak komunikacji między procesami** – wymiana halo jest lokalna/pomijalna. Izoluje sam narzut wątków.
- Wyraźne minimum przy **8 wątkach** (~8,1 s wobec 9,8 s dla 1 wątku).
- Czas obliczeń/krok **spada** dla 1→8 wątków (0,98 → 0,81 s), potem lekko rośnie przy 16 (nadsubskrypcja ponad liczbę rdzeni fizycznych).
- **Wniosek:** w obrębie pojedynczego procesu jądro obliczeniowe skaluje się wątkami poprawnie. Potwierdza to, że słabe skalowanie z rys. 3 wynika z komunikacji między locale (sieć), a nie z kodu obliczeniowego.

5. Skalowanie wątków, 1000³ na węźle .13 (1 locale, Chapel 2.9 mpi-conduit, 10 kroków)

Skalowanie wątków, 1000³ (1 locale, Chapel 2.9 mpi-conduit, węzeł .13: 24 rdzenie, 31 GB, 0 swap)



- Powtórzenie rys.4 na dużej kostce i dedykowanym węźle .13 (24 rdzenie, 31 GB, 0 swap). 1 locale, zmienne wątki 1 → 16.
- Przy 24 rdzeniach `maxTaskPar` = wartości żądanej dla *każdej* konfiguracji (brak ograniczania qthreads) – widać to wprost w logu (`[cfg] ... maxTaskPar=`).
- Duży skok 1 → 2 wątki, minimum przy **4 wątkach** (~5,60 s), powyżej – wolniej.
- Przyspieszenie płaszczy się na ~1,55×, mocno poniżej ideału; wydajność równoległa spada z 1,0 do ~0,09 **przy 16 wątkach**.
- **Wniosek:** jądro jest *ograniczone pamięcią* – powyżej 4 wątków dokładanie rdzeni nie pomaga (pasmowo wysyczone), a narzut synchronizacji/NUMA lekko pogarsza wynik. Kontrast z rys. 4 (500³, box 12-rdz.): tam za mało pracy na krok, więc 1 wątek był najszybszy; tu 8× większa kostka amortyzuje narzut i optimum przypada na 4 wątki.

Metodyka: dla rys.1–4 liczba wątków nie jest zapisana w logach – odtworzono ją z kolejności czasowej przebiegów (segmentacja DP), więc etykiety 1/2/4/8/16 są dedukowane; rozmiar N wyliczono z $\text{round}(\sqrt[3]{\text{sum}})$, a liczba locale jest jedyną wartością zapisaną wprost (`across N locale(s)`). Rys.5 pochodzi z nowszych przebiegów, które *jawnie* logują konfigurację (`[cfg] threadsPerLocale(requested)=` ze skryptu oraz `[cfg] locale <id> maxTaskPar=` z programu), więc nie wymaga żadnej dedukcji.